UNIVERSIDAD TECNOLÓGICA DE XICOTEPEC DE JUÁREZ

INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN E INNOVACIÓN DIGITAL

Asignatura:	Bases de Datos en la Nube
Nombre de la Práctica:	Examen Práctico: Instalación y Administración de PostgreSQL y MySQL en un VPS
Estudiante:	Mario Banda Sánchez
Matrícula:	240597
Grupo:	7° B
Equipo:	Bitwatt
Docente Evaluador:	Marco Antonio Ramírez Hernández
Proveedor VPS:	IP: 5.183.9.45
Puertos Asignados:	PostgreSQL: 40597 | MySQL: 40598 | pgAdmin: 40596
Fecha de Entrega:	26 de Octubre de 2026

Contenido

DESCRIPCIÓN DE LA PRÁCTICA 3
3.1 Contexto de la Infraestructura 3
3.2 Objetivo General 3
3.3 Tecnologías e Herramientas Utilizadas 3
3.4 Alcance del Ejercicio 3
DESARROLLO TÉCNICO 4
4.1 Adquisición y Configuración General del VPS 4
4.2 Gestión de Usuarios a Nivel de Sistema Operativo 4
4.3 Instalación de Herramientas y Docker Compose 4
4.4 Configuración de Puertos y Conexiones Remotas 5
4.5 Roles, Privilegios y Cuentas Personales 6
4.6 Objetos Relacionales (Tablas, Triggers y Vistas) 6
PRUEBAS Y RESULTADOS 6
5.1 Caso 01: Conexión Remota SSH e Inspección del VPS 6
5.2 Caso 02: Clonación del Repositorio Privado mediante PAT (Personal Access Token) 7
5.3 Caso 03: Despliegue de Servicios con Docker Compose y Diagnóstico 10
5.4 Caso 04: Configuración y Registro del Servidor en pgAdmin 4 Web 12
5.5 Caso 05: Inserción en PostgreSQL y Disparo de Trigger Auditoría 14
5.6 Caso 06: Actualización de Registros en PostgreSQL y Auditoría 16
5.7 Caso 07: Eliminación de Registros en PostgreSQL y Auditoría 17
5.8 Caso 08: Conexión Remota a MySQL desde MySQL Workbench 19
5.9 Caso 09: Creación de Usuarios de Equipo y Asignación GRANT en MySQL 20
5.10 Caso 10: Ingesta Masiva, Tablas y Triggers en MySQL 20
CONTROL DE VERSIONES Y REPOSITORIO GITHUB 23
6.1 Creación de la Rama de Trabajo ManualInstalacion-1uep 23
6.2 Incorporación de Entregables Obligatorios 23
6.3 Historial de Commits y Fusión (Merge) a Rama Principal 24
CONCLUSIONES PERSONALES 24
7.1 Competencias Desarrolladas 24
7.2 Dificultades Técnicas Encontradas 25
7.3 Soluciones Implementadas 25
7.4 Reflexión sobre Bases de Datos Relacionales en la Nube 25

DESCRIPCIÓN DE LA PRÁCTICA

3.1 Contexto de la Infraestructura

En entornos empresariales y de desarrollo moderno, la transición hacia arquitecturas de bases de datos relacionales en la nube mediante Servidores Privados Virtuales (VPS) y contenedores ha optimizado los procesos de despliegue, aislamiento de servicios y mantenibilidad. Esta práctica se desarrolla dentro de un VPS compartido por nuestro grupo en la Universidad Tecnológica de Xicotepec de Juárez, corriendo Ubuntu Linux accesible vía IP pública 5.183.9.45.

3.2 Objetivo General

Evaluar y demostrar las competencias prácticas del estudiante para desplegar, configurar, administrar y verificar de forma remota dos motores de bases de datos relacionales (PostgreSQL 18 y MySQL Server 8.0) junto con la interfaz web pgAdmin 4, utilizando Docker Compose en un VPS compartido, aplicando aislamiento de puertos por matrícula (240597), control de acceso basado en roles, triggers de auditoría y control de versiones con GitHub.

3.3 Tecnologías e Herramientas Utilizadas

- **Servidor Nube (VPS):** Ubuntu Linux Server 24.04 LTS en IP 5.183.9.45.
- **Contenedorización:** Docker Engine v26+ y Docker Compose v3.8.
- **Motores de BBDD:** PostgreSQL 18 y MySQL Server 8.0.
- **Clientes de Gestión:** pgAdmin 4 Web (puerto 40596) y MySQL Workbench 8.0.
- **Control de Versiones:** Git CLI y GitHub Repository (Flujo Git-flow con ramas dedicadas).
- **Acceso Remoto:** OpenSSH Client para administración del sistema operativo.

3.4 Alcance del Ejercicio

El alcance comprende desde el aprovisionamiento de la cuenta de usuario mario.banda en el SO del VPS, la orquestación independiente del stack multi-motor utilizando la matrícula 240597 para derivar los puertos remotos (PostgreSQL: 40597, MySQL: 40598), la definición de esquelética relacional (tb_users, tb_products, tb_logs), implementación de automatismos mediante Triggers SQL y Vistas, configuración de 7 roles e-commerce y 5 cuentas personales, la ejecución de 10 casos de prueba funcionales y el empaquetado final en el repositorio Git bajo la rama Manual_VPS_BDs_Relacionales.

DESARROLLO TÉCNICO

4.1 Adquisición y Configuración General del VPS

Se habilitó un Servidor Privado Virtual (VPS) accesible en Internet a través de la dirección IP pública 5.183.9.45. Se verificó el correcto funcionamiento del sistema operativo Ubuntu Linux, comprobando los recursos disponibles de CPU, memoria RAM y espacio en disco SSD.

4.2 Gestión de Usuarios a Nivel de Sistema Operativo

Para garantizar la auditoría y separación de responsabilidades entre los integrantes del equipo Bitwatt, se crearon cuentas de usuario dedicadas a nivel de sistema operativo. Para el desarrollo individual, se configuró la cuenta mario.banda con directorio de trabajo en /home/mario.banda/workspace, agregándolo a los grupos con privilegios sudo y docker.

4.3 Instalación de Herramientas y Docker Compose

Se verificaron las instalaciones del sistema de control de versiones Git, el motor Docker Engine y la herramienta Docker Compose. Se preparó un proyecto docker-compose.yml completamente propio e independiente de los demás integrantes del equipo, definiendo volúmenes persistentes nombrados postgres_data_240597 y mysql_data_240597 para prevenir la pérdida de datos.

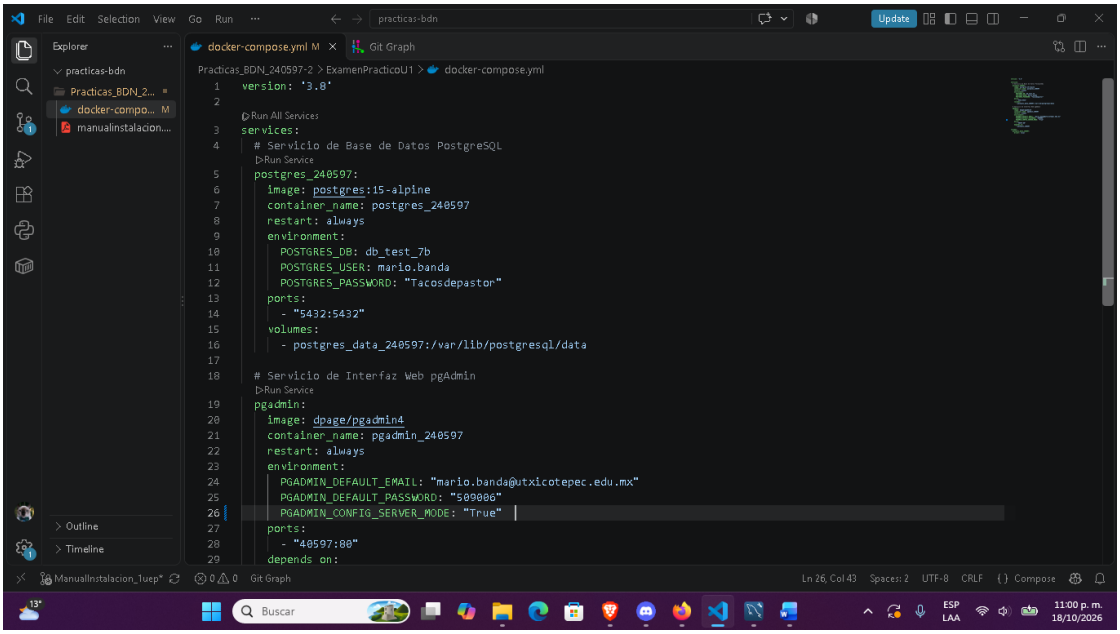


Figura: Archivo docker-compose.yml propio configurado en VS Code con puertos y volúmenes por matrícula.

Código Completo de docker-compose.yml:

```
version: '3.8'

services:
  postgres_240597:
    image: postgres:15-alpine
```

```
    container_name: postgres_240597
    restart: always
    environment:
      POSTGRES_DB: db_test_7b
      POSTGRES_USER: mario.banda
      POSTGRES_PASSWORD: "Tacosdepastor"
    ports:
      - "40597:5432"
    volumes:
      - postgres_data_240597:/var/lib/postgresql/data

  mysql_240597:
    image: mysql:8.0
    container_name: mysql_240597
    restart: always
    environment:
      MYSQL_DATABASE: db_test_7b
      MYSQL_ROOT_PASSWORD: "Tacosdepastor"
      MYSQL_USER: mario.banda
      MYSQL_PASSWORD: "Tacosdepastor"
    ports:
      - "40598:3306"
    volumes:
      - mysql_data_240597:/var/lib/mysql

  pgadmin_240597:
    image: dpage/pgadmin4
    container_name: pgadmin_240597
    restart: always
    environment:
      PGADMIN_DEFAULT_EMAIL: "mario.banda@utxicotepec.edu.mx"
      PGADMIN_DEFAULT_PASSWORD: "Tacosdepastor"
      PGADMIN_CONFIG_SERVER_MODE: "True"
    ports:
      - "40596:80"
    depends_on:
      - postgres_240597

volumes:
  postgres_data_240597:
  mysql_data_240597:
```

4.4 Configuración de Puertos y Conexiones Remotas

Para evitar colisiones entre contenedores en el VPS compartido, los puertos mapeados externamente se derivaron de los 5 dígitos de la matrícula del estudiante (240597):

- Puerto PostgreSQL: 40597 (Publicado sobre el puerto TCP interno 5432).
- Puerto MySQL: 40598 (Publicado sobre el puerto TCP interno 3306, calculado como matrícula + 1).
- Puerto pgAdmin Web: 40596 (Publicado sobre el puerto HTTP interno 80).

4.5 Roles, Privilegios y Cuentas Personales

En ambos motores de base de datos se implementaron los 7 roles requeridos para el contexto e-commerce: superadmin, admin, seller, buyer, support, common y not_registered. Asimismo, se crearon las 5 cuentas personales utilizando datos de integrantes del equipo con sus respectivas asignaciones de roles y permisos estrictos de lectura/escritura/administración.

4.6 Objetos Relacionales (Tablas, Triggers y Vistas)

Se construyó la base de datos db_test_7b conteniendo las tablas relacionales tb_users, tb_products y tb_logs para el registro automático de auditoría.

Estructura de la Tabla tb_logs (Auditoría):

```
CREATE TABLE IF NOT EXISTS tb_logs (
    id INT AUTO_INCREMENT PRIMARY KEY,
    table_name VARCHAR(80) NOT NULL,
    table_operation VARCHAR(10) NOT NULL,
    db_user VARCHAR(80) NOT NULL,
    operation_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    table_description TEXT
);
```

Se implementaron Triggers SQL para interceptar eventos de inserción, actualización y eliminación, registrando automáticamente en tb_logs la operación realizada, la estampa de tiempo y el usuario actuante. Finalmente, se diseñó la vista de trazabilidad vw_audit_trail para consultar de forma rápida la historia de transacciones.

PRUEBAS Y RESULTADOS

A continuación se documentan detalladamente los diez casos de prueba funcionales ejecutados sobre la infraestructura en la nube, incluyendo las sentencias SQL/Bash, la descripción del procedimiento, las capturas de pantalla de evidencia y el resultado verificado.

5.1 Caso 01: Conexión Remota SSH e Inspección del VPS

• Descripción: Establecimiento de una sesión remota segura mediante SSH al VPS desde una terminal cliente, verificando la IP pública 5.183.9.45 y la autenticación del usuario mario.banda.

• Comando de Ejecución:

```
ssh mario.banda@5.183.9.45
```

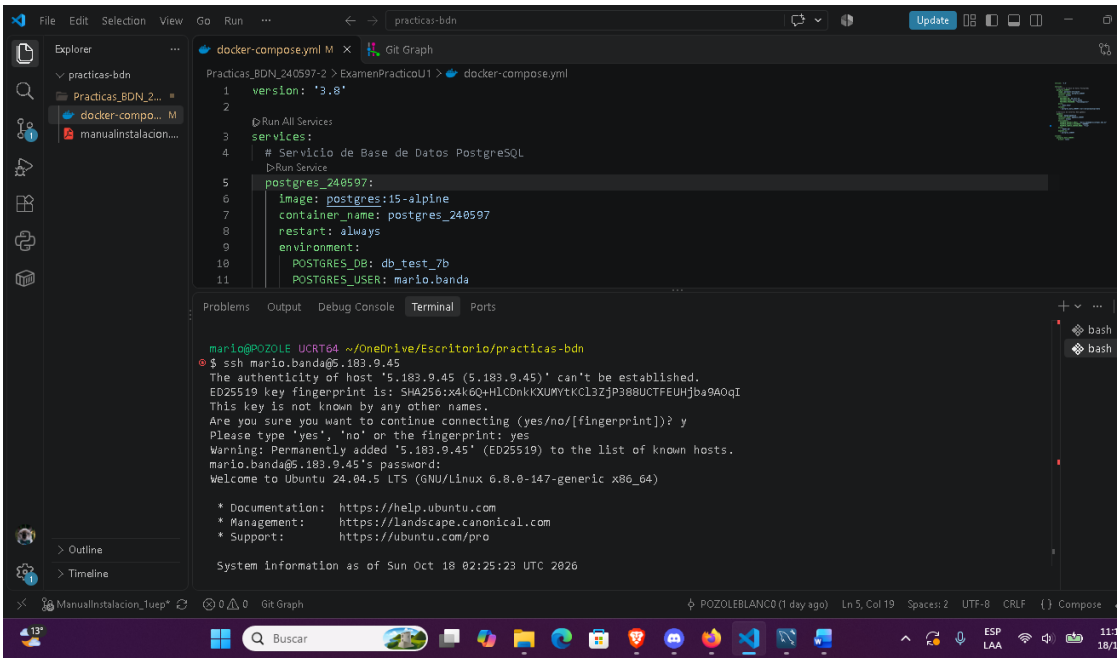


Figura: Caso 01 - Inicio de sesión SSH exitoso en el servidor VPS Ubuntu Linux (IP: 5.183.9.45).

• Resultado Obtención: Conexión SSH establecida satisfactoriamente. Acceso concedido al directorio /home/mario.banda/.

5.2 Caso 02: Clonación del Repositorio Privado mediante PAT (Personal Access Token)

• Descripción: Generación de un Personal Access Token (PAT Classic) en GitHub con permisos 'repo' y clonación del repositorio privado de la materia dentro de la zona de trabajo propia /home/mario.banda/workspace/.

• Sentencia / Comando:

```
cd /home/mario.banda/workspace/
git clone https://github.com/POZOLEBLANC0/Practicas_BDN_240597-2.git
```

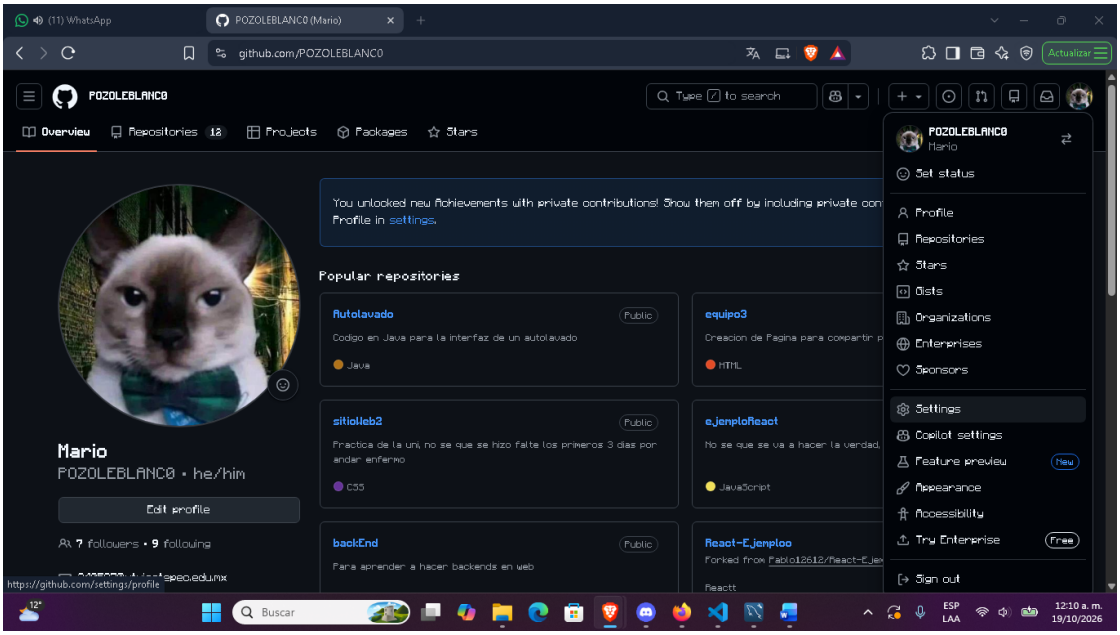


Figura: Caso 02 - Acceso a la configuración de perfil en GitHub.

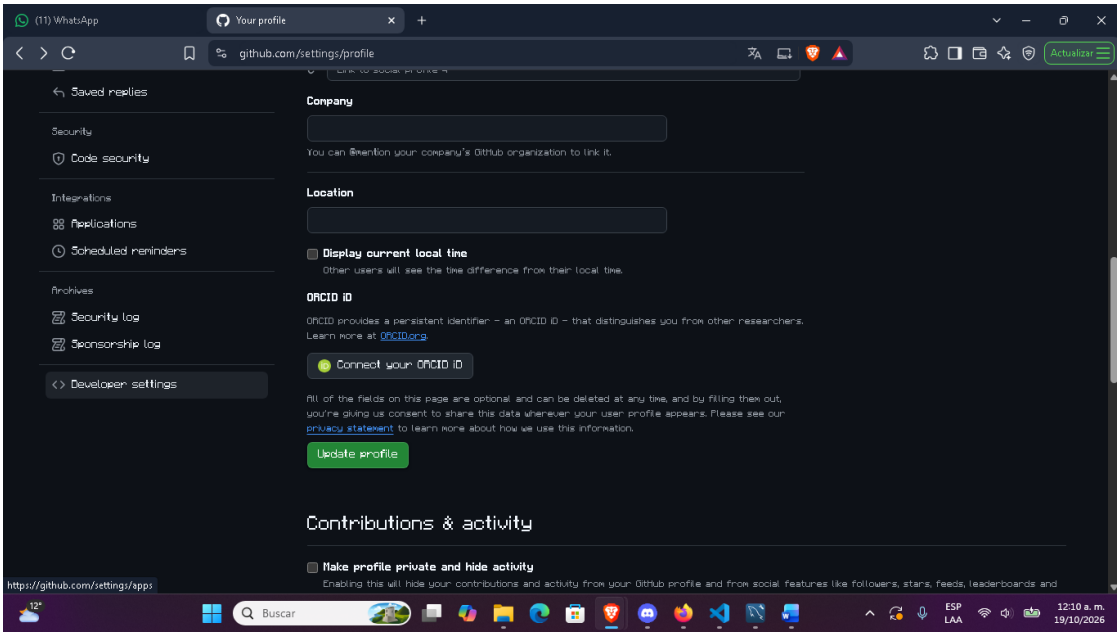


Figura: Caso 02 - Interfaz de GitHub Developer Settings para Tokens de Acceso.

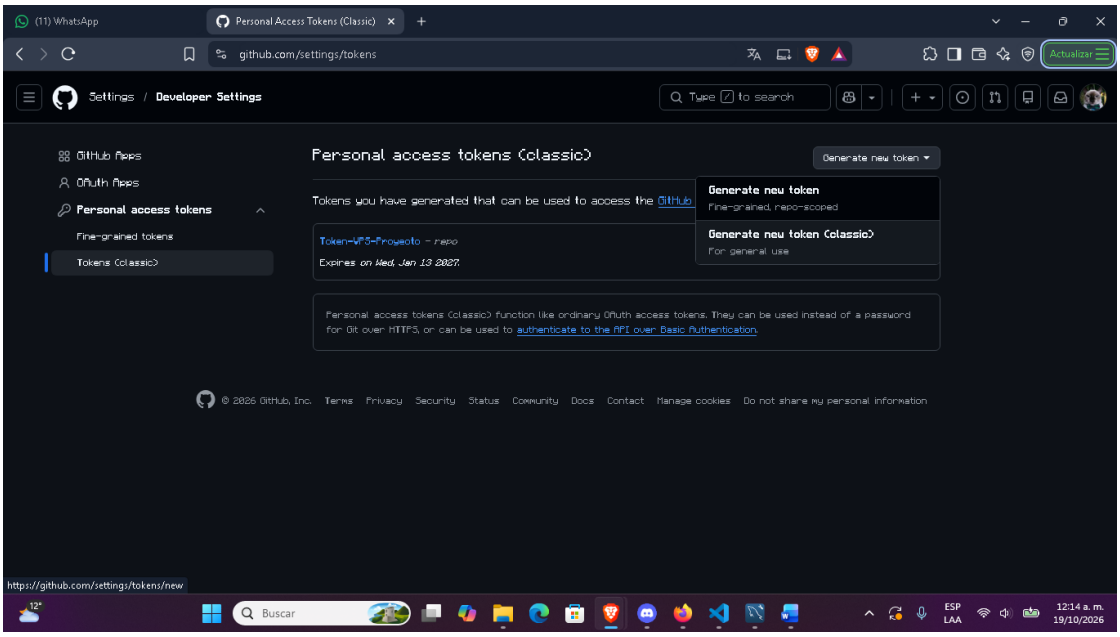


Figura: Caso 02 - Selección de Personal Access Tokens (Classic).

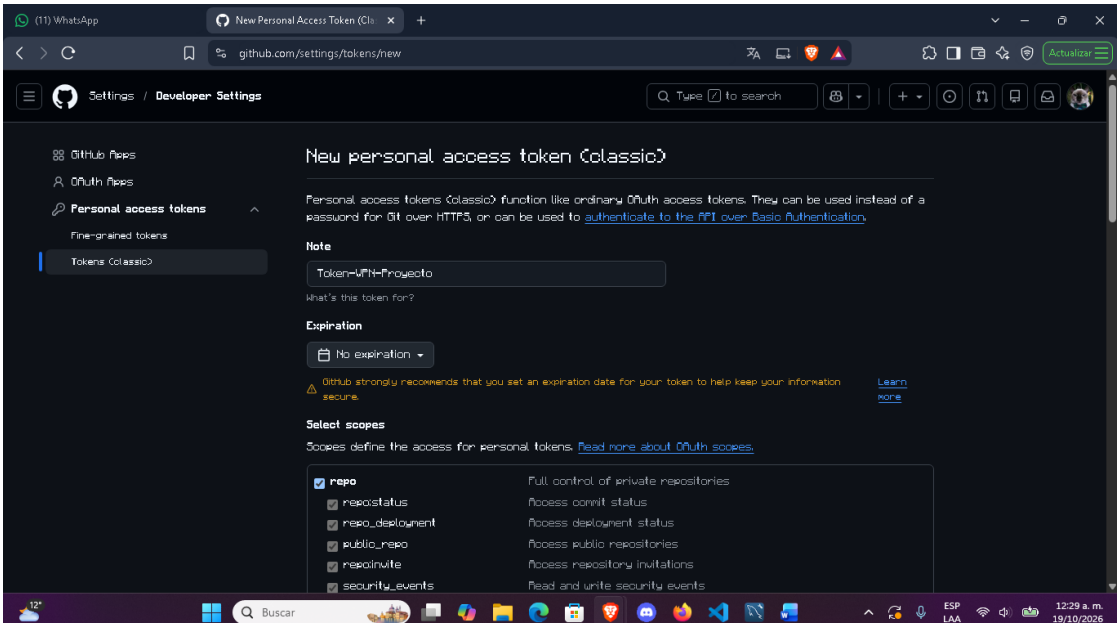


Figura: Caso 02 - Configuración de alcance 'repo' y generación del token de acceso.

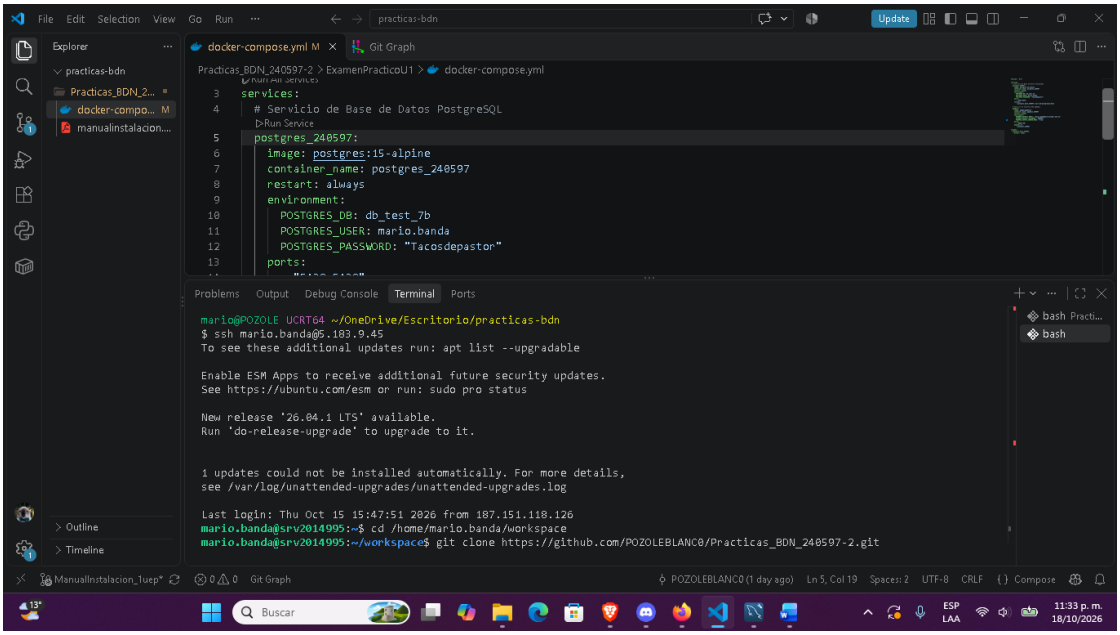


Figura: Caso 02 - Navegación al directorio workspace en el VPS.

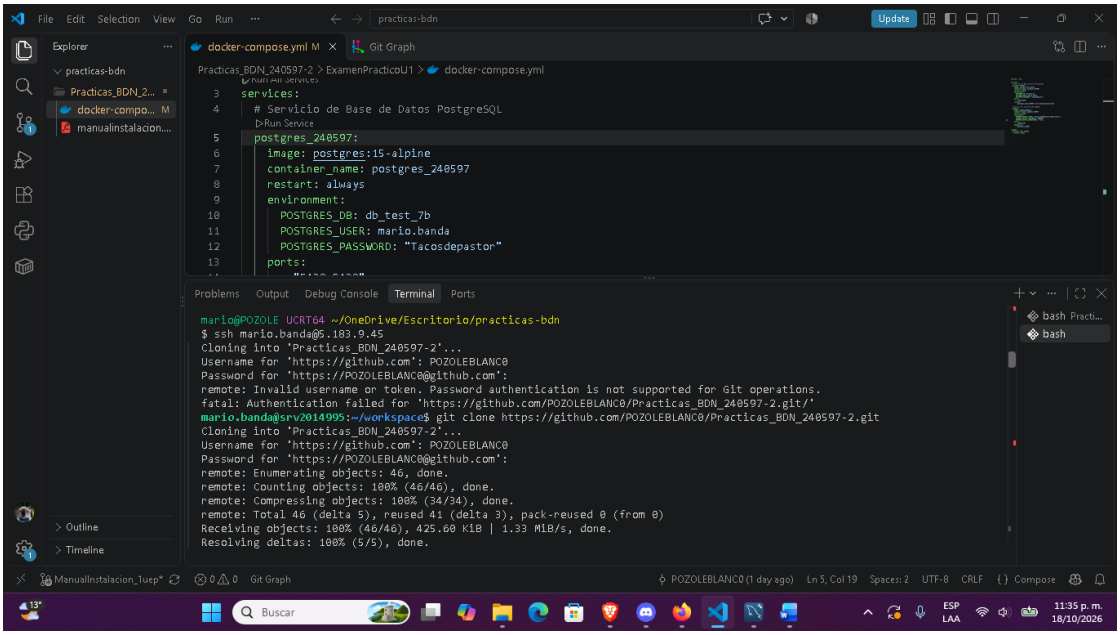


Figura: Caso 02 - Clonación remota del repositorio Git utilizando el token de autenticación PAT.

• Resultado Obtención: Repositorio clonado exitosamente localmente en el VPS con el árbol de archivos completo.

5.3 Caso 03: Despliegue de Servicios con Docker Compose y Diagnóstico

• Descripción: Ejecución en segundo plano de los contenedores postgres_240597, mysql_240597 y pgadmin_240597 utilizando docker compose up -d, seguida de la verificación del estado de ejecución y mapeo de puertos con docker ps.

• Comando de Ejecución:

```
cd /home/mario.banda/workspace/Practicas_BDN_240597-2/ExamenPracticoU1/
docker compose up -d
docker ps
```

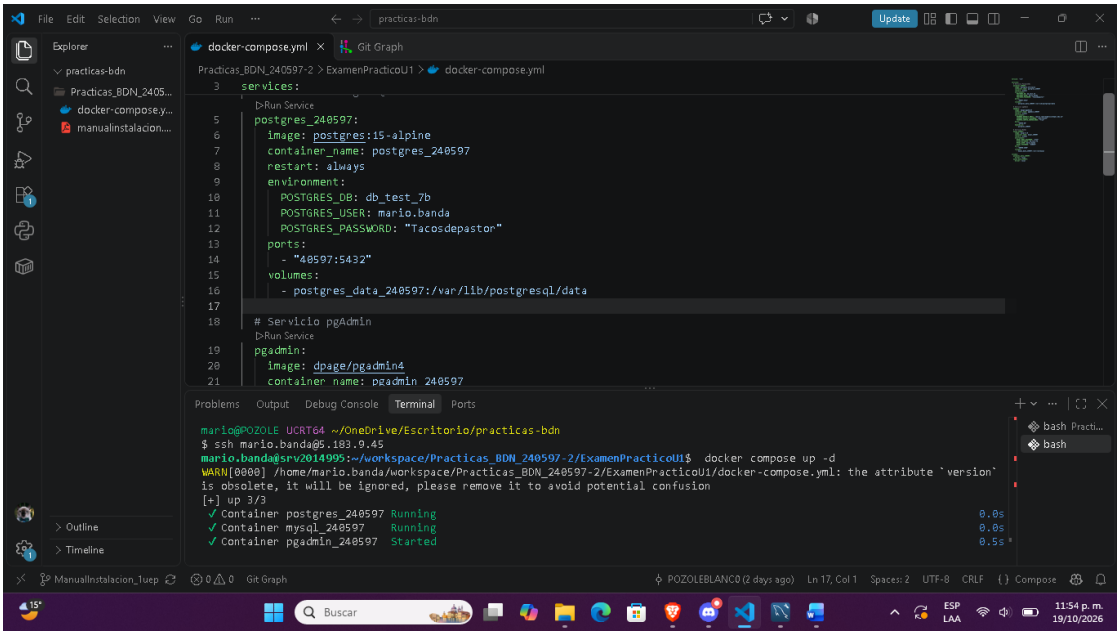


Figura: Caso 03 - Levantamiento exitoso de los tres contenedores con Docker Compose.

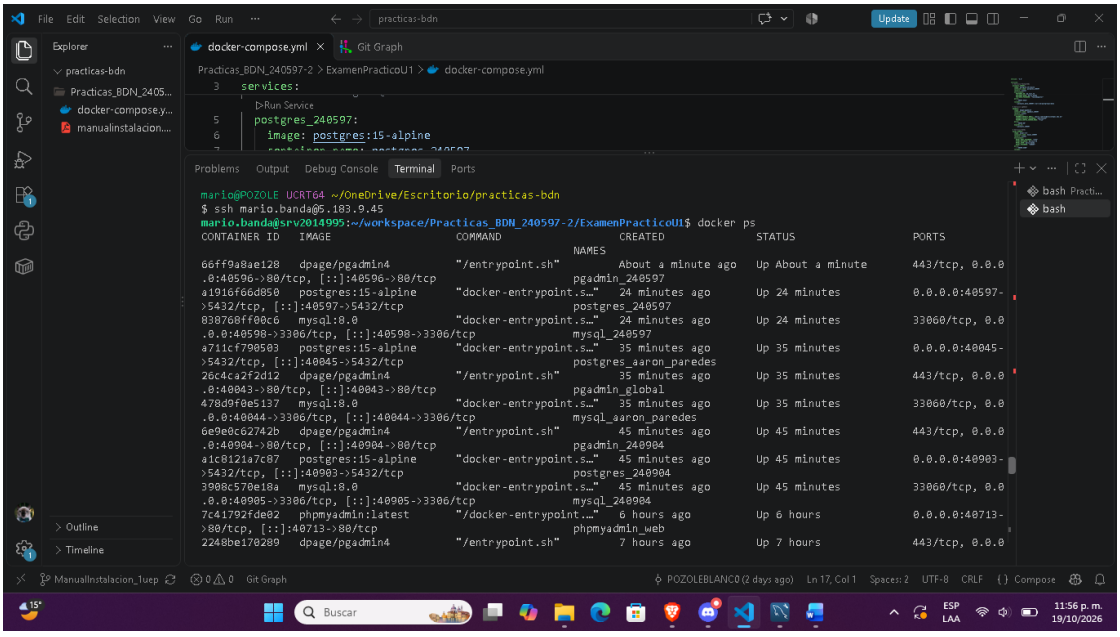


Figura: Caso 03 - Inspección con 'docker ps' confirmando contenedores en ejecución y puertos 40596, 40597 y 40598.

• Resultado Obtención: Contenedores inicializados sin errores, en estado Running y publicando los puertos personalizados de forma independiente.

5.4 Caso 04: Configuración y Registro del Servidor en pgAdmin 4 Web

• Descripción: Acceso a la interfaz web de pgAdmin 4 a través del puerto 40596 y vinculación del servidor PostgreSQL postgres_240597 configurando credenciales, puerto interno 5432 y la base de datos db_test_7b.

• Parámetros de Conexión: Hostname: postgres_240597 | Port: 5432 | DB: db_test_7b | User: mario.banda

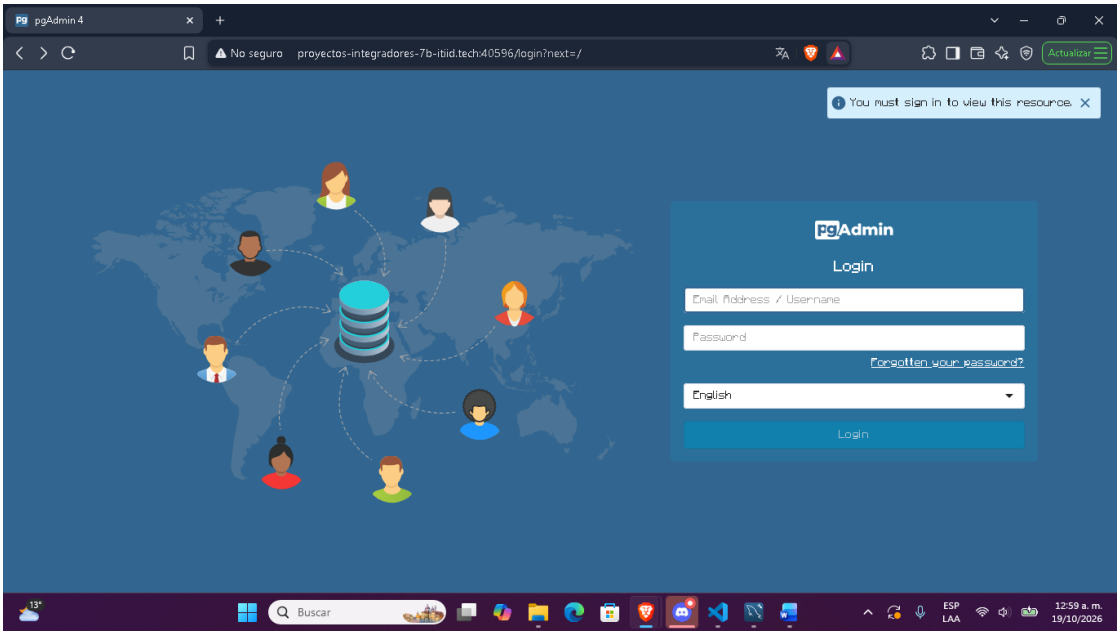


Figura: Caso 04 - Pantalla de inicio de sesión de pgAdmin 4 Web (puerto 40596).

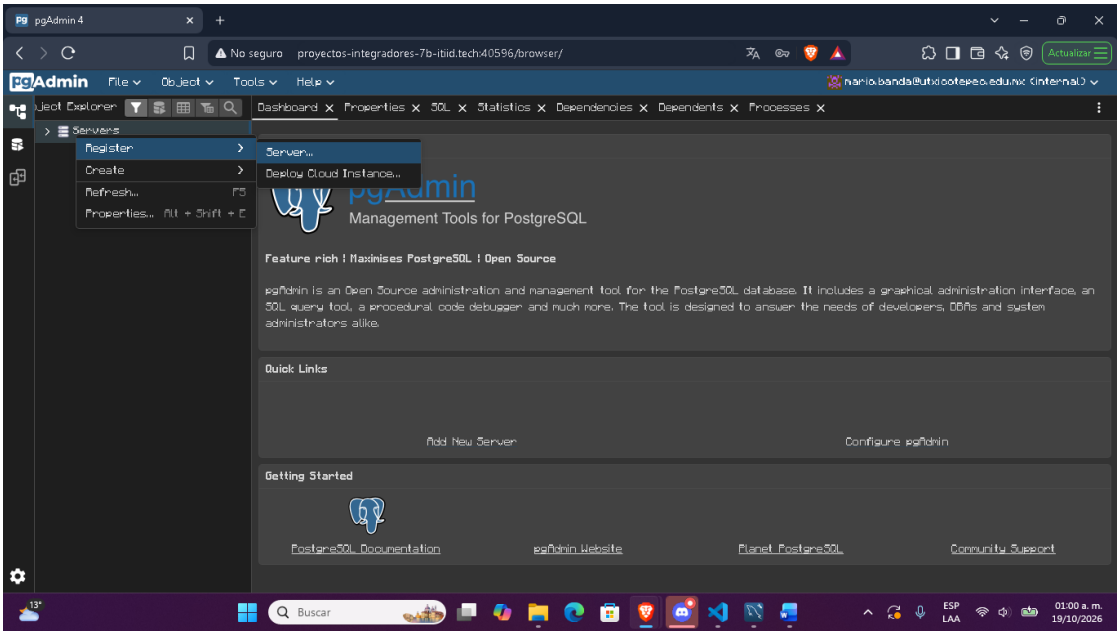


Figura: Caso 04 - Menú de registro de nuevo servidor en pgAdmin.

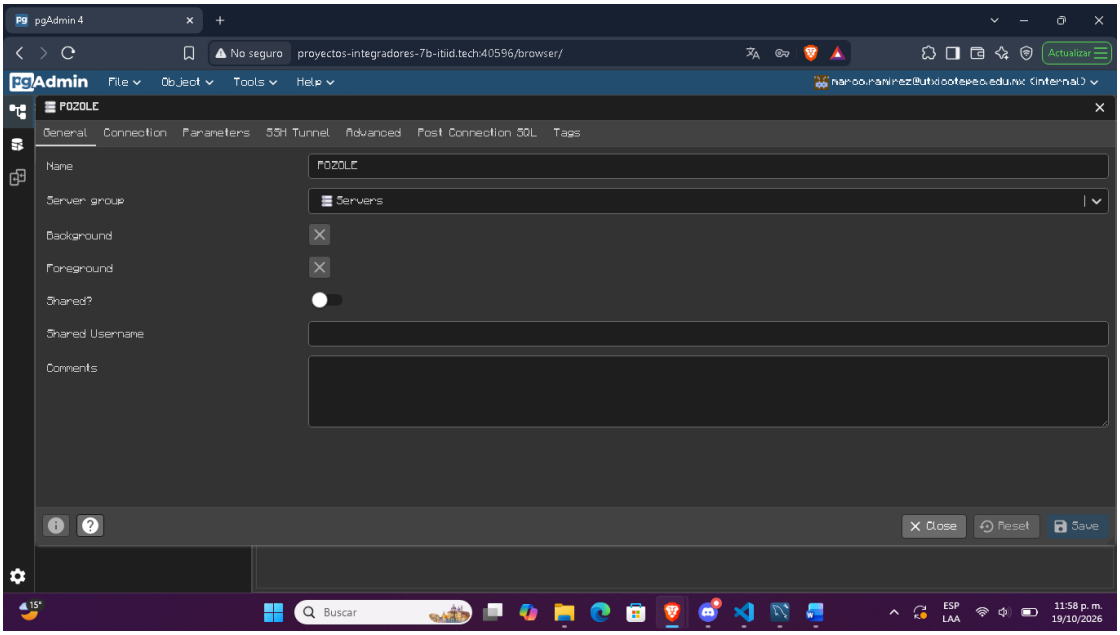


Figura: Caso 04 - Pestaña General definiendo el nombre del servidor 'POZOLE'.

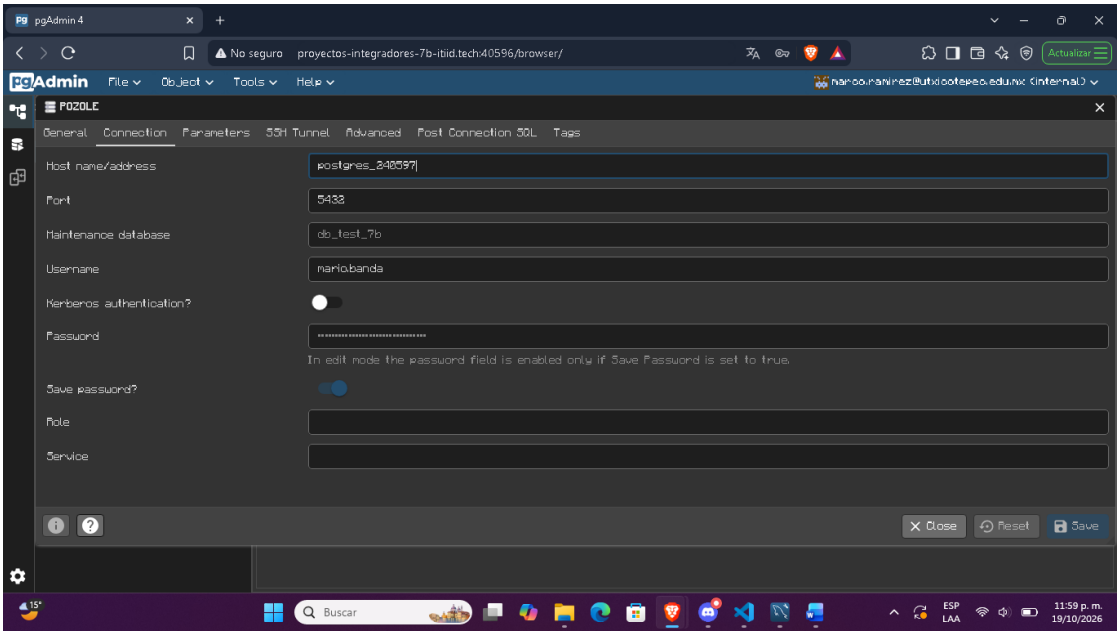


Figura: Caso 04 - Pestaña Connection especificando host postgres_240597, puerto 5432 y usuario mario.banda.

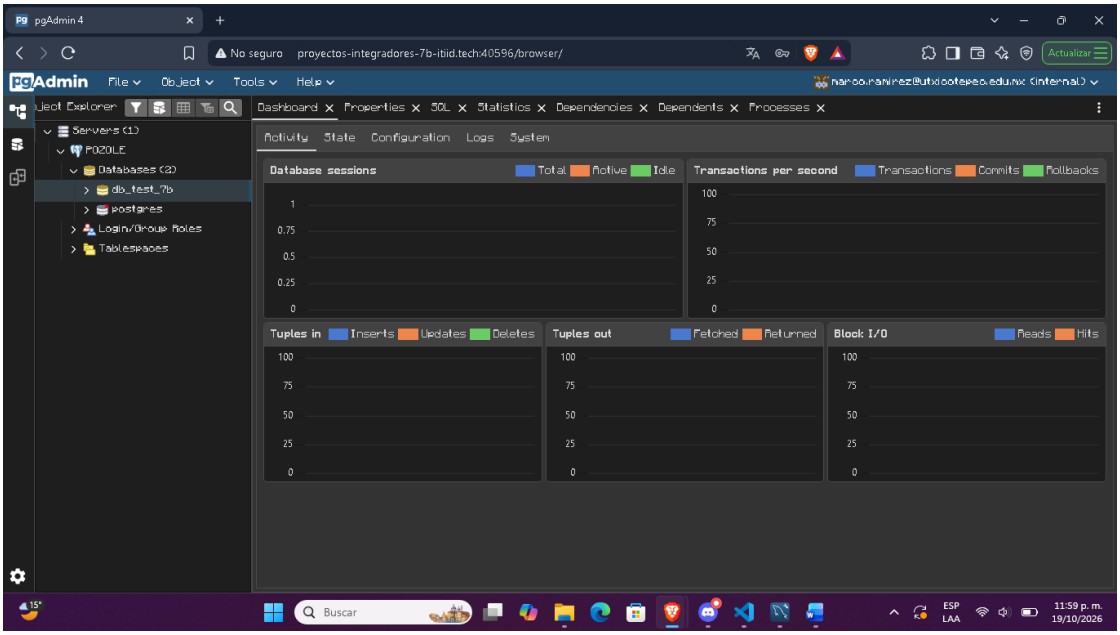


Figura: Caso 04 - Dashboard de pgAdmin 4 mostrando métricas activas de la conexión a PostgreSQL.

• Resultado Obtención: Conexión remota establecida y servidor registrado en pgAdmin 4, permitiendo la exploración de esquemas y ejecución de queries.

5.5 Caso 05: Inserción en PostgreSQL y Disparo de Trigger Auditoría

• Descripción: Registro de tres nuevos usuarios en la tabla tb_users en PostgreSQL y verificación de la generación automática de los logs de auditoría en la tabla tb_logs mediante el Trigger trg_users_audit.

• Sentencias SQL:

```
-- Inserción de usuarios
INSERT INTO tb_users (email, nick, password) VALUES
('edith@gmail.com', 'miamor', '1234'),
('gadiel@gmail.com', 'gadi', '1234'),
('samu@gmail.com', 'samepil', '1234');

-- Verificación de auditoría
SELECT * FROM tb_logs;
```

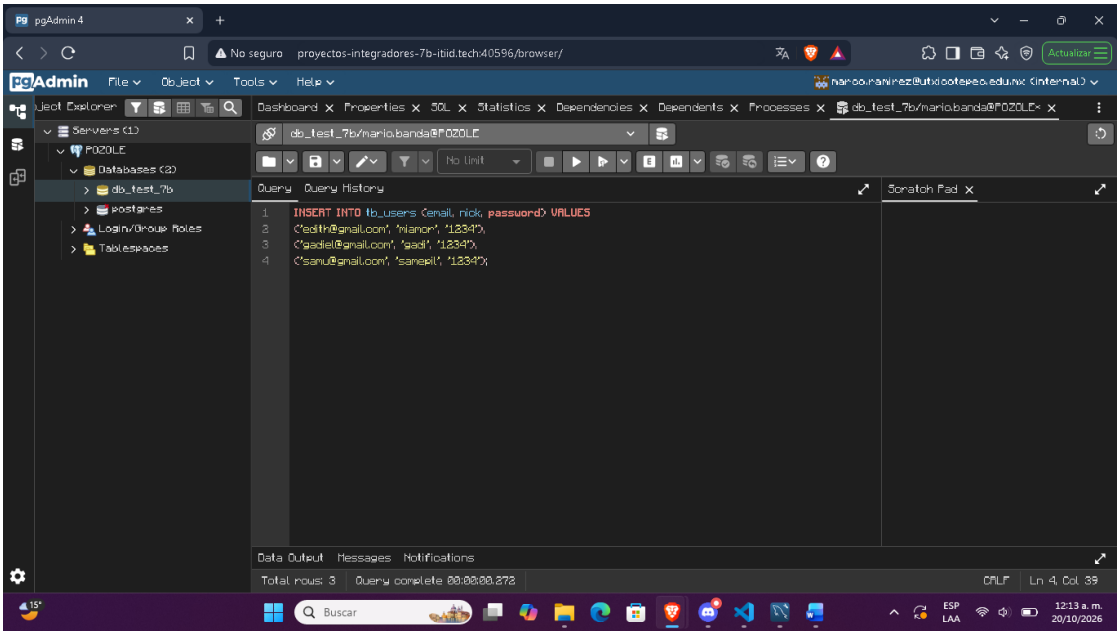


Figura: Caso 05 - Sentencia SQL INSERT ejecutada para 3 usuarios en pgAdmin 4.

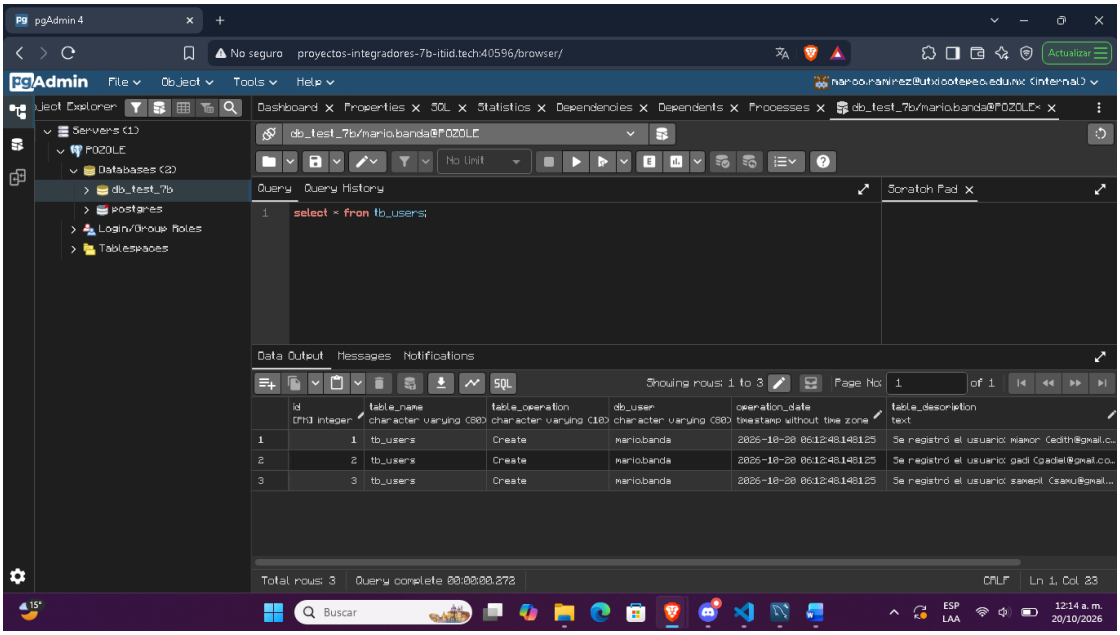


*Figura: Caso 05 - Consulta SELECT * FROM tb_users confirmando los 3 registros creados.*

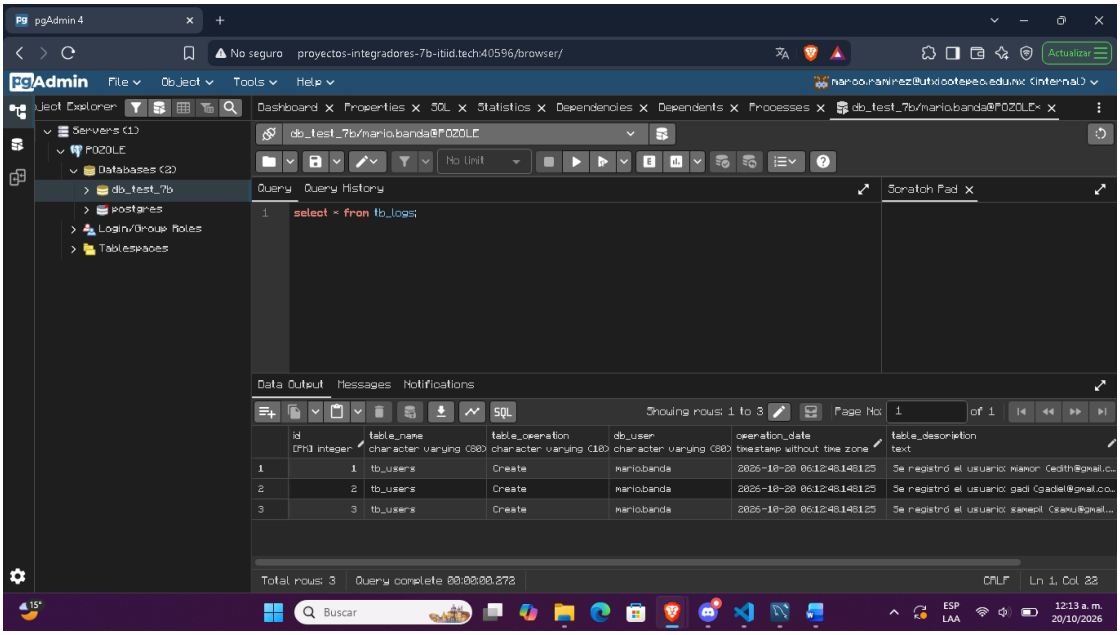


*Figura: Caso 05 - Consulta SELECT * FROM tb_logs verificando los 3 eventos 'Create' capturados por el trigger.*

• Resultado Obtención: Inserción exitosa de usuarios y disparo inmediato del Trigger registrando las operaciones 'Create' con usuario mario.banda y timestamp exacto.

5.6 Caso 06: Actualización de Registros en PostgreSQL y Auditoría

• Descripción: Modificación de los campos nick y email de un usuario en tb_users y comprobación de la captura de la operación 'Update' en tb_logs.

• Sentencias SQL:

```
UPDATE tb_users
SET nick = 'samepil', email = 'samuel.vargas@gmail.com'
WHERE id = 3;

SELECT * FROM tb_logs;
```

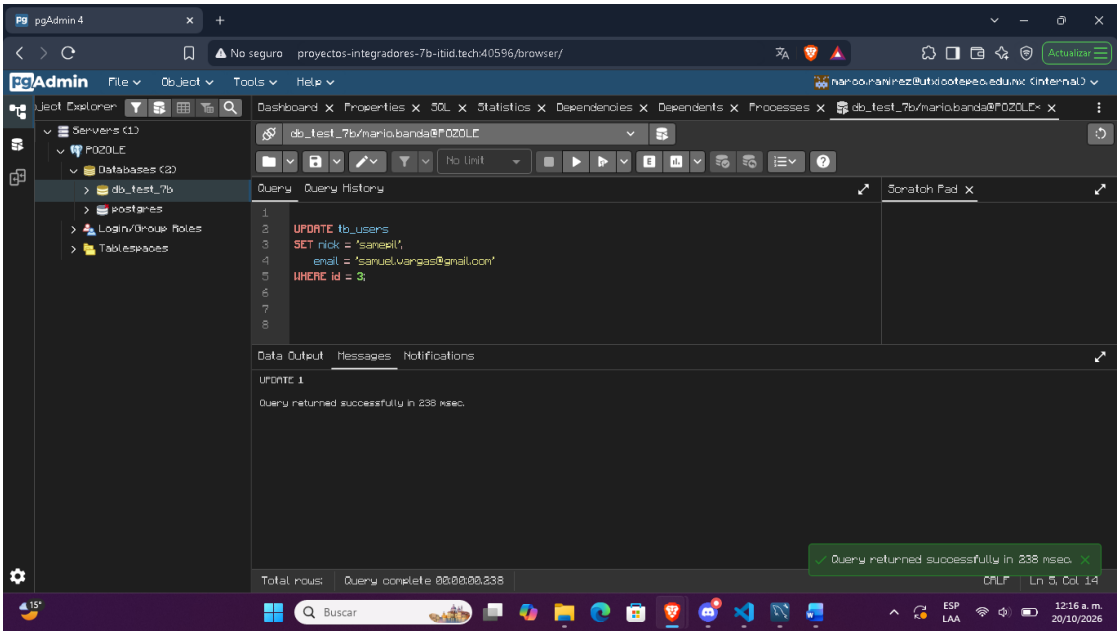


Figura: Caso 06 - Ejecución de sentencia UPDATE modificando el usuario con ID 3.

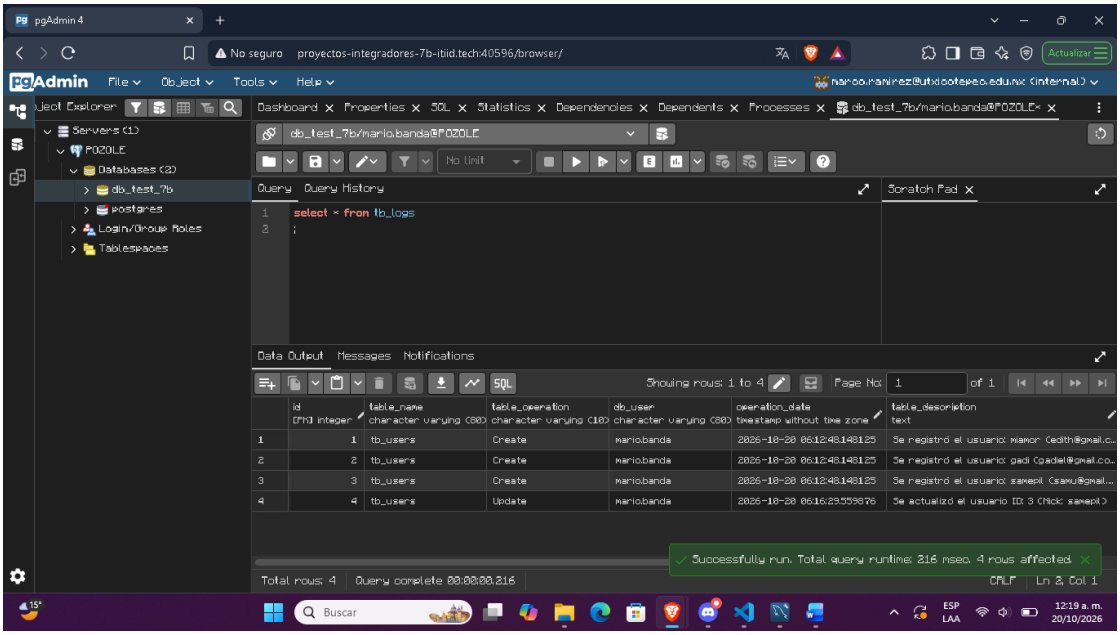


Figura: Caso 06 - Auditoría en tb_logs mostrando 4 registros (3 Create y 1 Update).

• Resultado Obtención: Fila actualizada correctamente y nuevo registro de log de auditoría insertado automáticamente con el detalle 'Se actualizó el usuario ID: 3'.

5.7 Caso 07: Eliminación de Registros en PostgreSQL y Auditoría

• Descripción: Eliminación de un usuario en tb_users y verificación de la entrada de trazabilidad con la operación 'Delete' en tb_logs.

• Sentencias SQL:

```
DELETE FROM tb_users WHERE id = 2;

SELECT * FROM tb_logs;
```

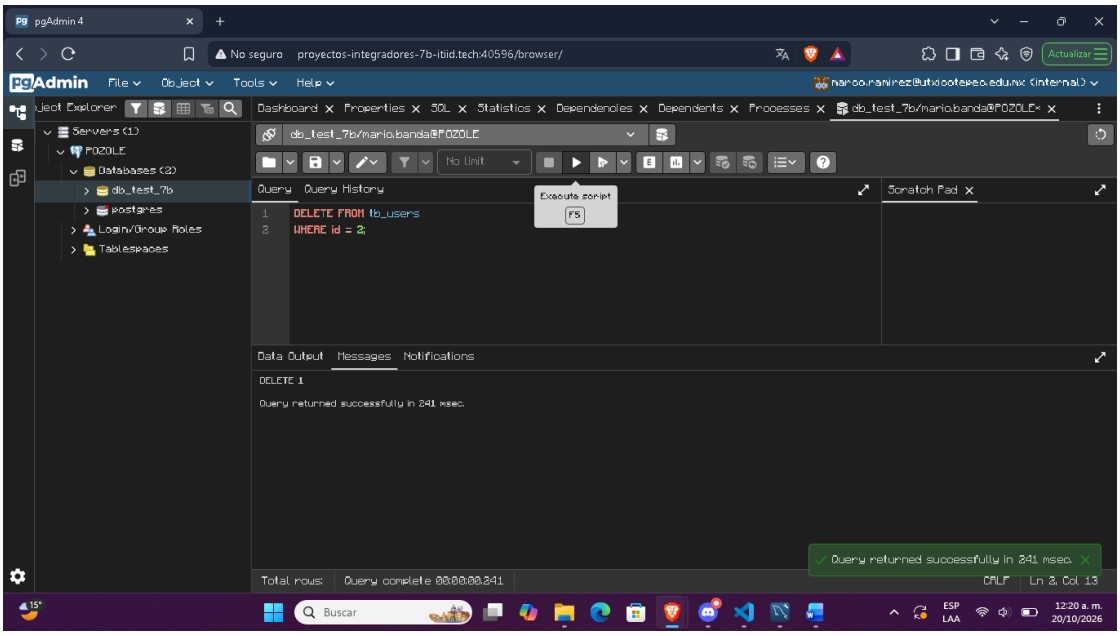


Figura: Caso 07 - Ejecución de la consulta DELETE FROM tb_users WHERE id = 2 en pgAdmin 4.

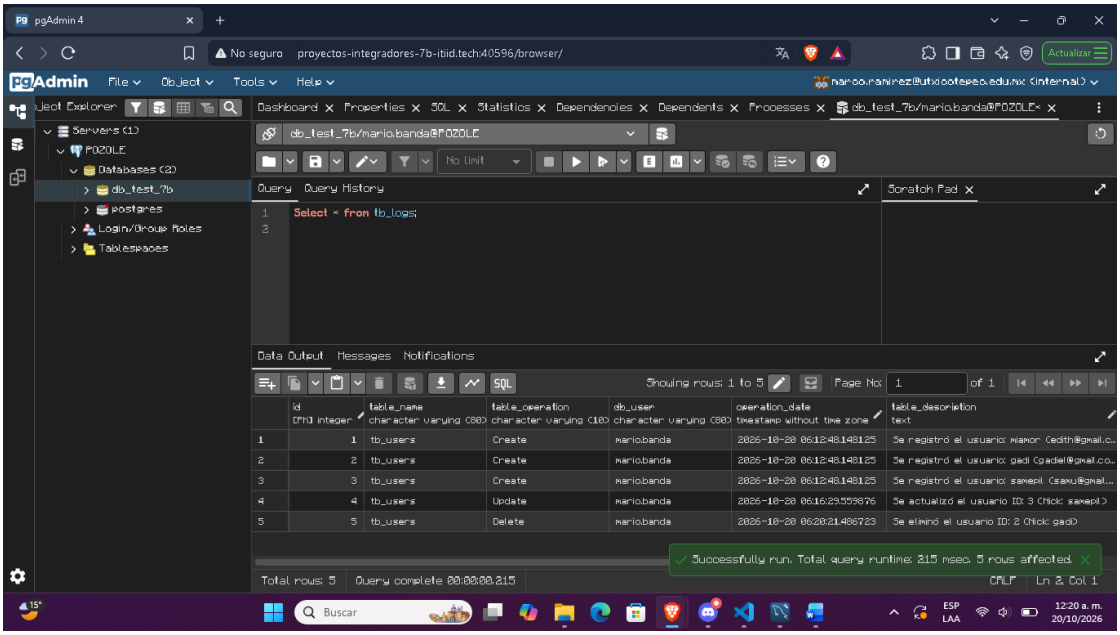


Figura: Caso 07 - Trazabilidad en tb_logs conteniendo 5 filas totales y el registro del evento Delete.

• Resultado Obtención: Usuario eliminado de la tabla principal y registrado automáticamente en el historial de auditoría.

5.8 Caso 08: Conexión Remota a MySQL desde MySQL Workbench

• Descripción: Configuración y prueba de conexión remota desde el cliente gráfico MySQL Workbench hacia el puerto personalizado 40598 publicado por el contenedor mysql_240597 en la IP 5.183.9.45.

• Parámetros de Conexión: Hostname: 5.183.9.45 | Port: 40598 | Username: mario.banda | Default Schema: db_test_7b

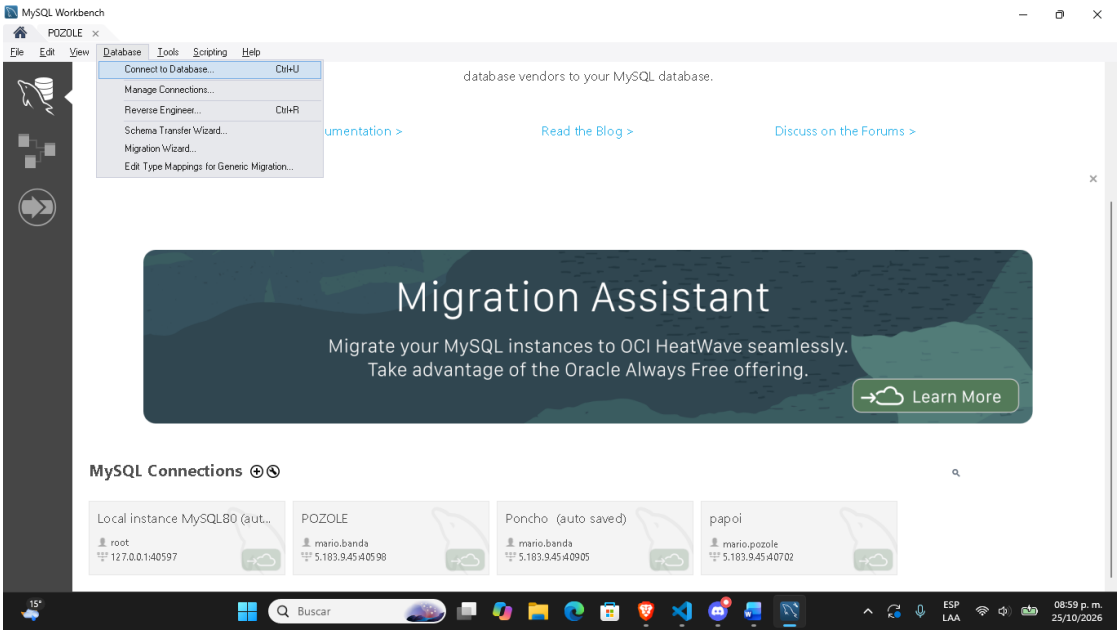


Figura: Caso 08 - Pantalla principal de MySQL Workbench listando la conexión 'POZOLE'.

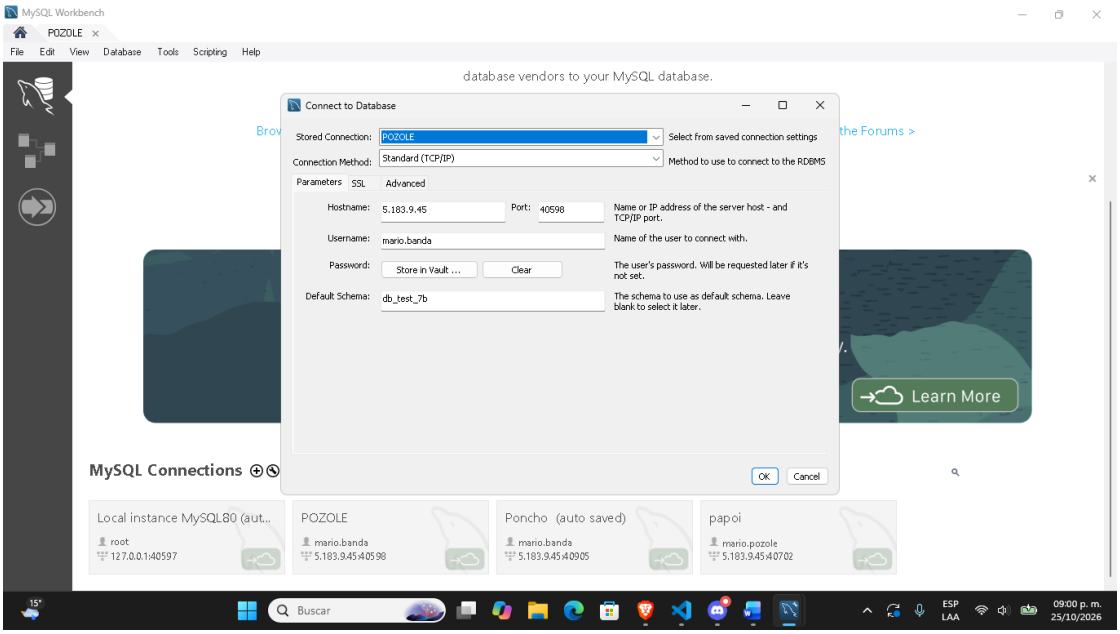


Figura: Caso 08 - Diálogo de configuración Connect to Database especificando IP 5.183.9.45 y puerto 40598.

• Resultado Obtención: Conexión remota a MySQL Server exitosa desde un cliente externo a la red de Docker.

5.9 Caso 09: Creación de Usuarios de Equipo y Asignación GRANT en MySQL

• Descripción: Creación de cuentas de usuario para compañeros del equipo en MySQL Server y asignación de privilegios de manipulación de datos (DML) sobre la base de datos db_test_7b.

• Sentencias SQL:

```
CREATE USER 'pablo'@'%' IDENTIFIED BY 'Pass123!';
GRANT SELECT, INSERT, UPDATE ON db_test_7b.* TO 'pablo'@'%';
FLUSH PRIVILEGES;

SELECT Host, User FROM mysql.user;
```

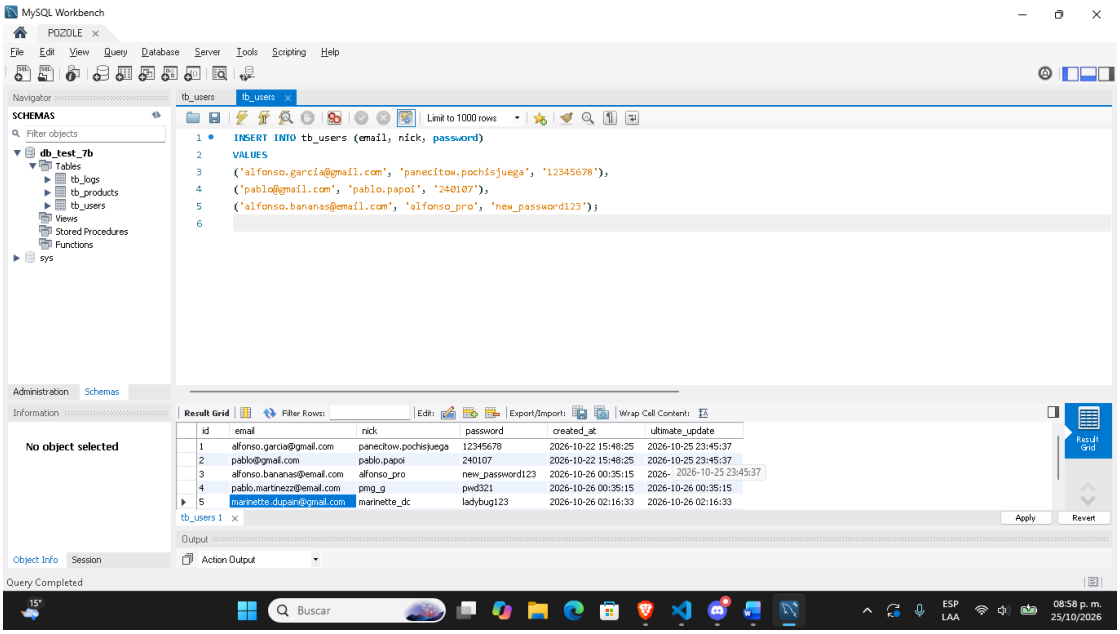


Figura: Caso 09 - Inserción de usuarios de prueba en MySQL Workbench y verificación de tabla tb_users.

• Resultado Obtención: Usuarios de equipo creados con permisos GRANT limitados según su rol asignado.

5.10 Caso 10: Ingesta Masiva, Tablas y Triggers en MySQL

• Descripción: Construcción de tablas tb_users, tb_products y tb_logs en MySQL Server, implantación de Triggers de auditoría para INSERT/UPDATE/DELETE e ingesta masiva de registros.

• Sentencias SQL (Triggers MySQL):

```
DELIMITER //
CREATE TRIGGER trg_users_after_insert AFTER INSERT ON tb_users
FOR EACH ROW BEGIN
  INSERT INTO tb_logs(table_name, table_operation, db_user, table_description)
  VALUES ('tb_users', 'Create', USER(), CONCAT('Se creó el usuario: ', NEW.nick));
END //
```

```
CREATE TRIGGER trg_users_after_update AFTER UPDATE ON tb_users
FOR EACH ROW BEGIN
  INSERT INTO tb_logs(table_name, table_operation, db_user, table_description)
  VALUES ('tb_users', 'Update', USER(), CONCAT('Se actualizó el usuario ID: ',
NEW.id));
END //

CREATE TRIGGER trg_users_after_delete AFTER DELETE ON tb_users
FOR EACH ROW BEGIN
  INSERT INTO tb_logs(table_name, table_operation, db_user, table_description)
  VALUES ('tb_users', 'Delete', USER(), CONCAT('Se eliminó el usuario: ', OLD.nick));
END //
DELIMITER ;
```

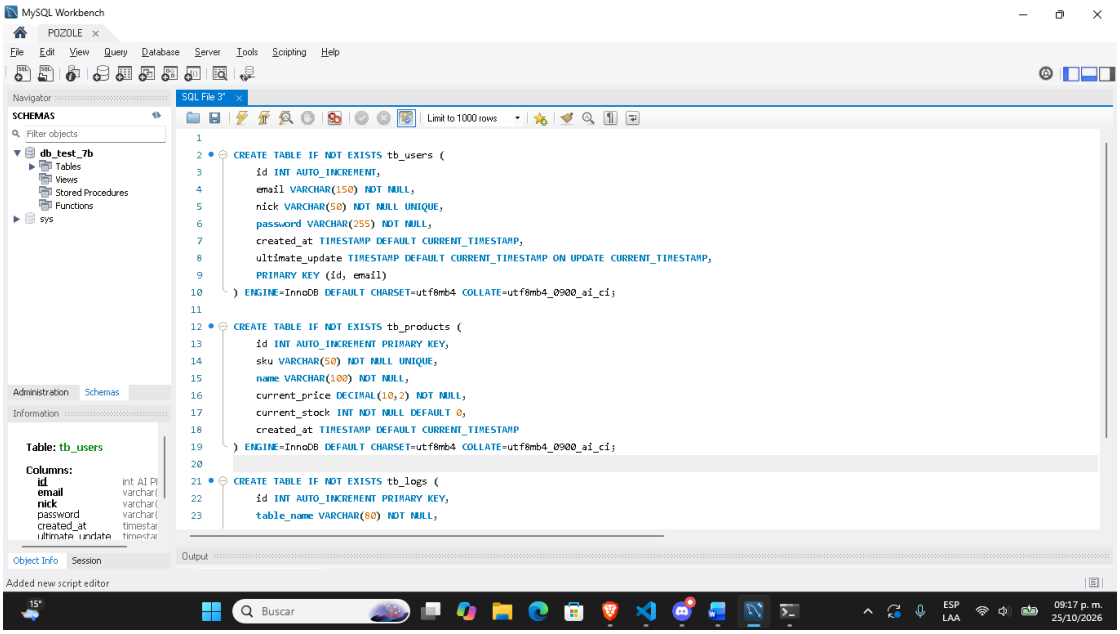


Figura: Caso 10 - Sentencias DDL de creación de tablas tb_users, tb_products y tb_logs en MySQL.

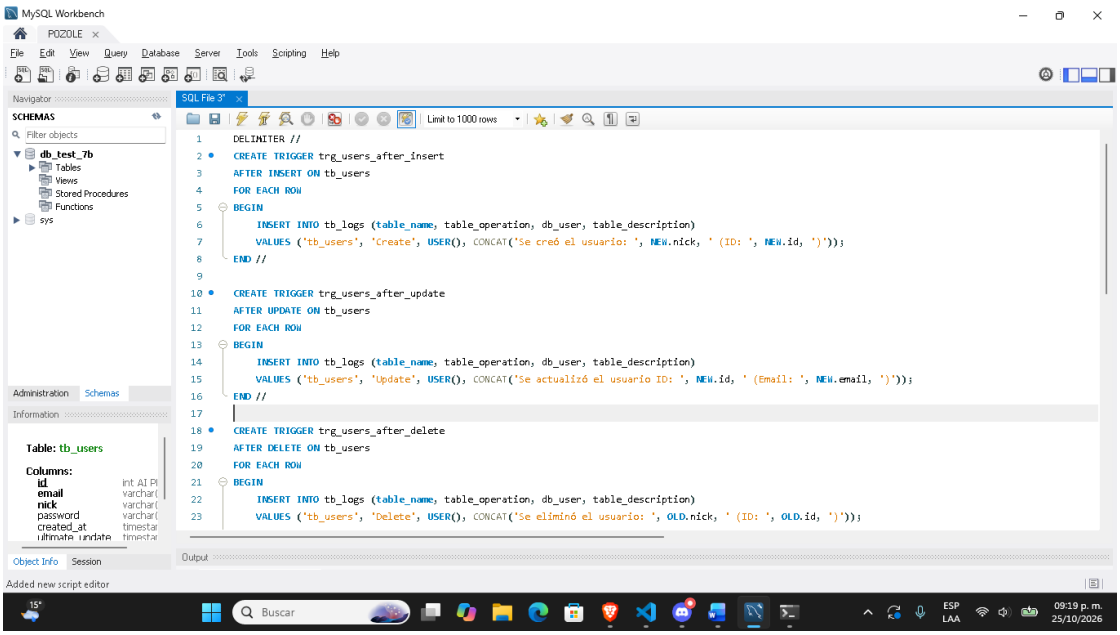


Figura: Caso 10 - Script SQL de definición de Triggers de auditoría en MySQL Workbench.

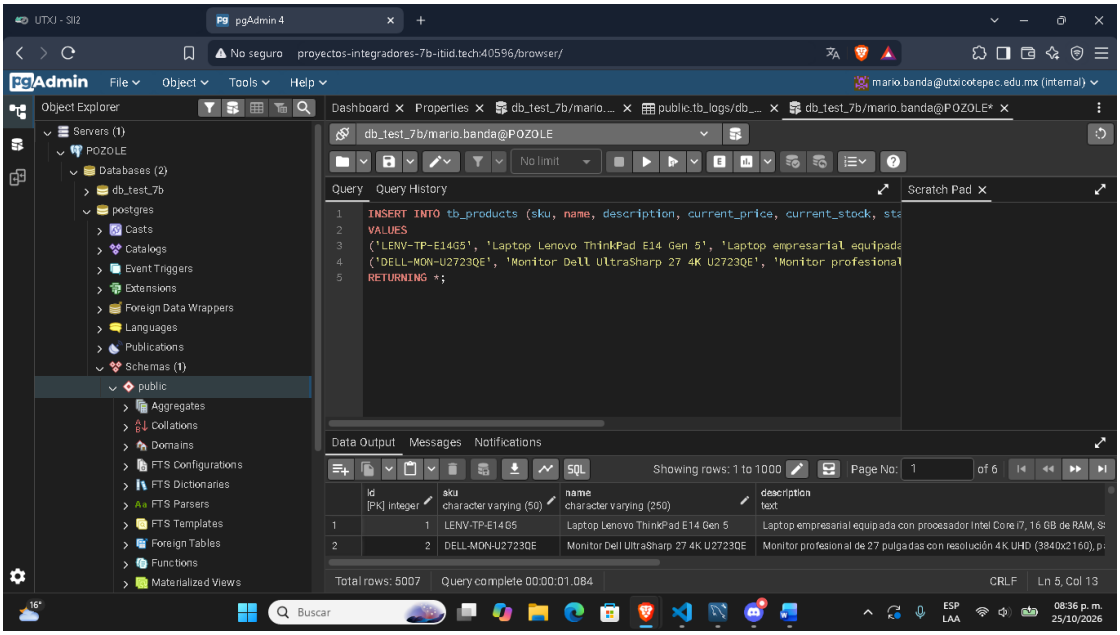


Figura: Caso 10 - Inserción de catálogo de productos en tb_products.

• Resultado Obtención: Tablas, Triggers y vistas creados en MySQL. Ingesta de datos verificada correctamente.

CONTROL DE VERSIONES Y REPOSITORIO GITHUB

El desarrollo técnico se administró rigurosamente mediante el sistema de control de versiones Git, garantizando la trazabilidad, reversibilidad y respaldo en la nube dentro del repositorio privado de GitHub.

6.1 Creación de la Rama de Trabajo ManualInstalacion-1uep

Para aislar los cambios del examen práctico de la rama principal, se creó y utilizó la rama de trabajo requerida por la rúbrica:

```
git checkout -b ManualInstalacion-1uep
# O alternativamente:
git switch -c ManualInstalacion-1uep
```

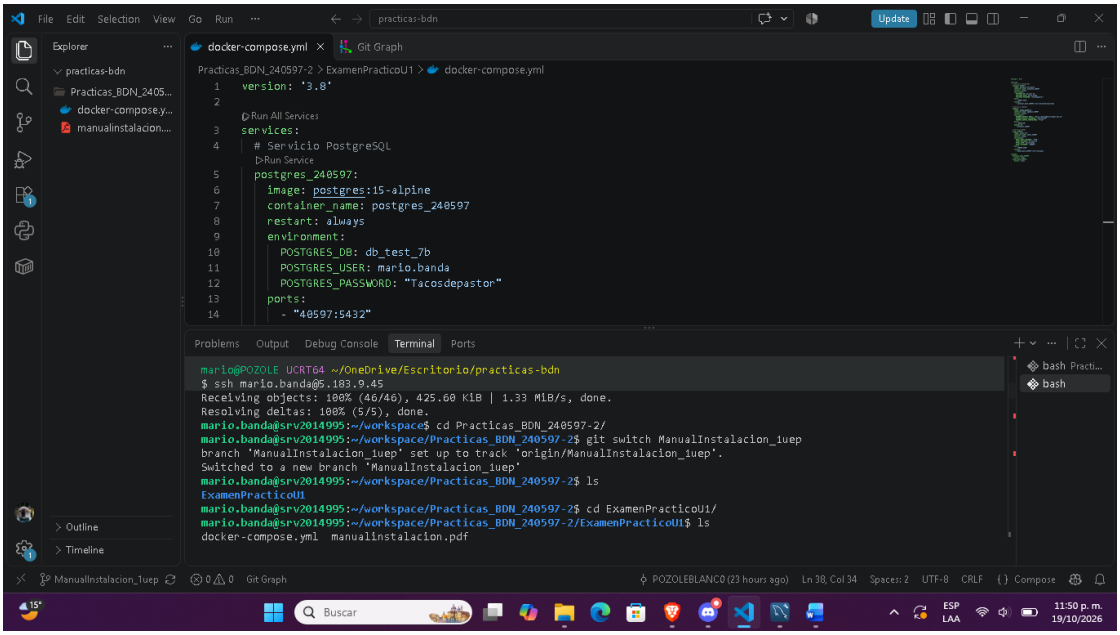


Figura: Control de Versiones - Creación y cambio a la rama ' ManualInstalacion-1uep' en el servidor VPS.

6.2 Incorporación de Entregables Obligatorios

Se incorporaron los archivos requeridos organizados bajo la siguiente estructura dentro de la rama:

```
ManualInstalacion-1uep /
├── Reporte_Unidad1_240597.pdf
├── docker-compose.yml
├── sql/
│   ├── postgresql.sql
│   └── mysql.sql
```

```
└── README.md
```

6.3 Historial de Commits y Fusión (Merge) a Rama Principal

Se realizaron commits periódicos con mensajes descriptivos sobre los avances y se ejecutó la sincronización con el repositorio remoto GitHub. Finalmente, se fusionó la rama de trabajo a la rama main sin conflictos.

```
git add .
git commit -m "Examen Practico de la Unidad 1 - Configuracion de docker-compose y
scripts SQL"
git push origin ManualInstalacion-1uep

# Fusión a rama principal
git checkout main
git merge ManualInstalacion-1uep
git push origin main
```

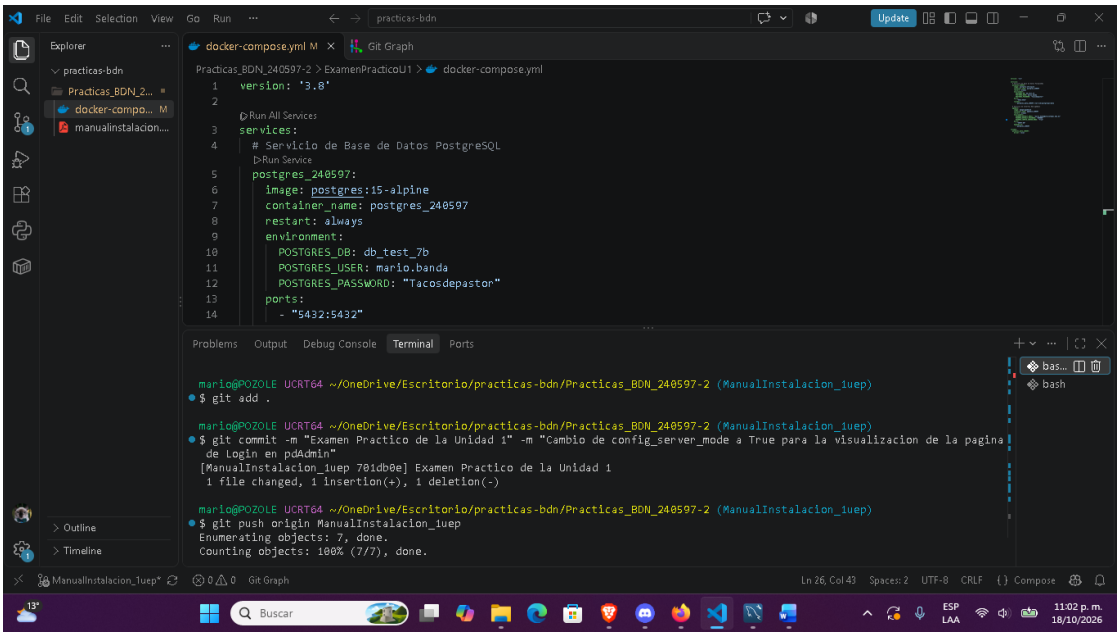


Figura: Control de Versiones - Ejecución de git add, git commit y git push desde VS Code hacia GitHub.

CONCLUSIONES PERSONALES

7.1 Competencias Desarrolladas

A lo largo de este examen práctico se logró consolidar el aprendizaje en el despliegue de infraestructura de bases de datos relacionales sobre servidores privados virtuales (VPS) en la nube. Se adquirieron

habilidades clave en orquestación con Docker Compose, aislamiento de puertos TCP por matrícula, configuración de seguridad basada en roles y creación automatizada mediante Triggers SQL.

7.2 Dificultades Técnicas Encontradas

1. Conflicto de Puertos en Entorno Compartido: Al compartir un mismo VPS entre múltiples integrantes del equipo, fue fundamental validar estrictamente que los puertos expuestos (40597 y 40598) no colisionaran con los contenedores de otros compañeros.
2. Persistencia de Datos en Contenedores: La posibilidad de perder la información al recrear o reiniciar un contenedor exigió la correcta configuración de volúmenes nombrados persistentes.
3. Autenticación Segura en Git: La migración de contraseñas convencionales a Tokens de Acceso Personal (PAT) en GitHub para repositorios privados requirió ajustar la configuración del cliente Git CLI en Linux.

7.3 Soluciones Implementadas

• Se utilizó una convención de nombres única incorporando la matrícula 240597 tanto en el nombre de los contenedores como en los volúmenes de datos del docker-compose.yml.
• Se agregó la variable de entorno PGADMIN_CONFIG_SERVER_MODE: 'True' en pgAdmin 4 para resolver problemas de autenticación web.
• Se verificaron las reglas del firewall y la apertura de direcciones IP de escucha (0.0.0.0) para garantizar conexiones remotas desde clientes externos como MySQL Workbench.

7.4 Reflexión sobre Bases de Datos Relacionales en la Nube

La experiencia práctica demostró que la administración de bases de datos relacionales en la nube abarca mucho más que la escritura de sentencias SQL. Requiere un dominio integral de arquitectura de servidores, redes, containerización y políticas de seguridad. La adopción de contenedores Docker simplifica drásticamente el despliegue y mantenimiento de infraestructuras de datos complejas, garantizando entornos homogéneos, portables y altamente eficientes para el desarrollo de software moderno.